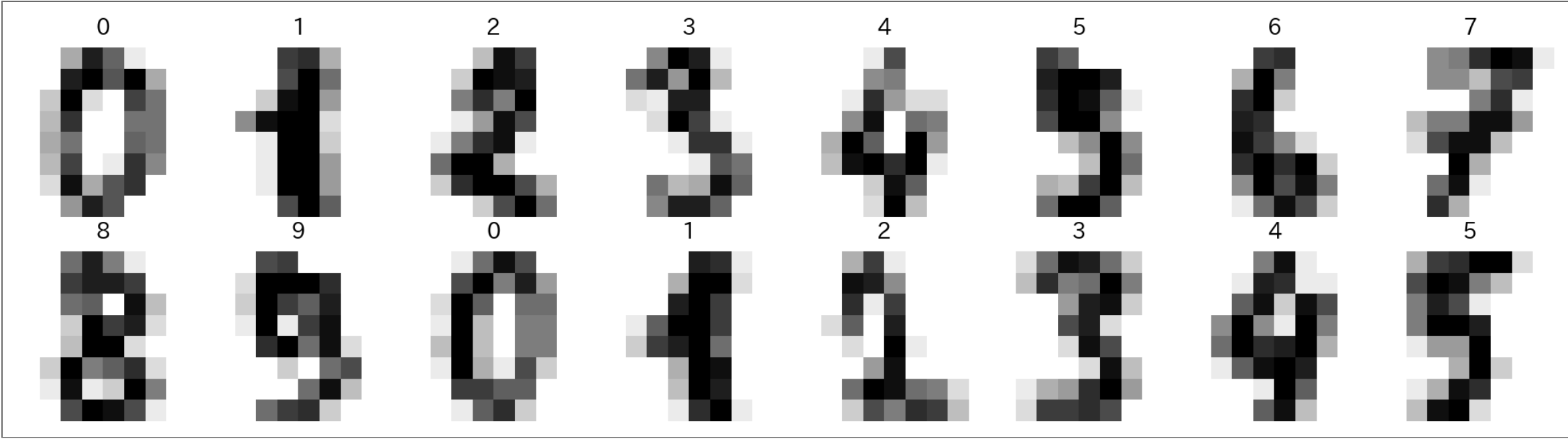


手書き数字分類プロジェクト

約束の回収

rkskmt

初回の約束、覚えているか



最終回までに、この仕組みを **中身から全部** 自分の手で組み立てる。ブラックボックスは残さない。

- 今日が最終回。 **手書き数字を読み取る機械** を完成させる
- 必要な部品は、ここまでの13回で **すべて自作済み**

データ：MNIST

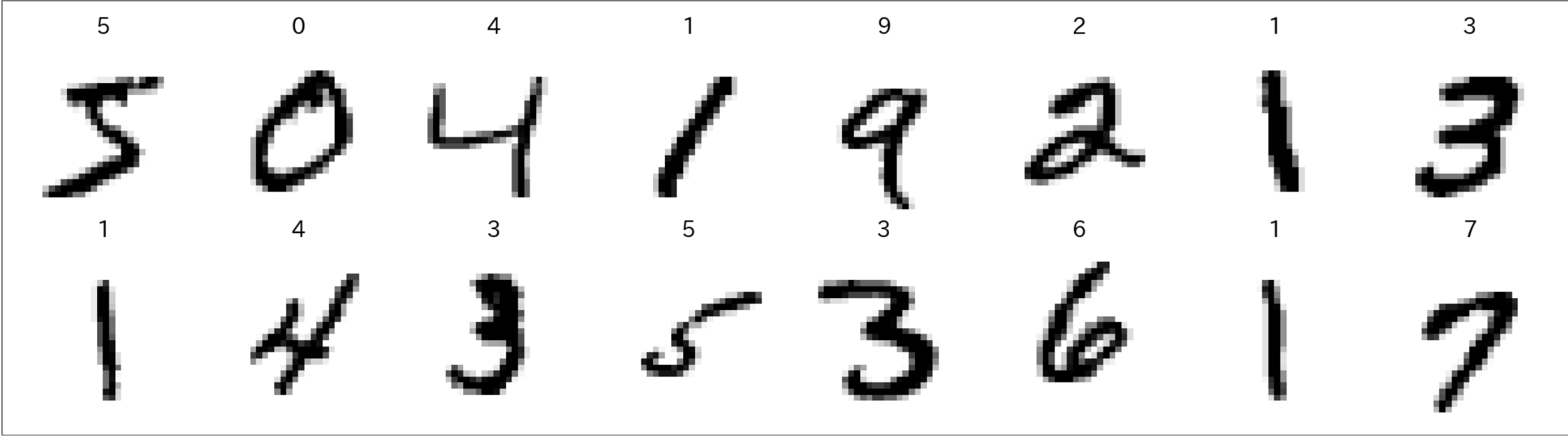
- 手書き数字の定番データセット。 **訓練6万枚・テスト1万枚**、各28×28ピクセル

Code

```
1 import torch
2 import torch.nn as nn
3 from torchvision import datasets, transforms
4
5 transform = transforms.ToTensor() # 画像 → 0~1 の tensor
6 train_ds = datasets.MNIST(root="data", train=True, download=True, transform=transform)
7 test_ds = datasets.MNIST(root="data", train=False, download=True, transform=transform)
8
9 print("訓練:", len(train_ds), "枚 / テスト:", len(test_ds), "枚")
10 img, label = train_ds[0]
11 print("1枚の形:", tuple(img.shape), " 正解ラベル:", label)
```

Result

訓練: 60000 枚 / テスト: 10000 枚
1枚の形: (1, 28, 28) 正解ラベル: 5



画像は784次元のベクトル

- $28 \times 28 =$ **784個の数字の並び** — モデルにとって画像は784次元のベクトル
- 魚は2次元（輝度・重さ）だった。 次元の呪いの回で遊んだ高次元 が、 **本物の入力** としてやってきた
- **nn.Flatten()** が「画像 → ベクトル」の変換をやる

① ノート

784次元でも、やることは魚の2次元と **完全に同じ**： 特徴量ベクトル → 変換を重ねる → 確率 → 損失 → 勾配降下。

出力層を付け替える：10クラス分類

- 答えは Salmon / Sea bass の2択ではなく、**0～9 の10択**
- 「変換を重ねる」の回の表 のとおり、**出力層を付け替える** だけ

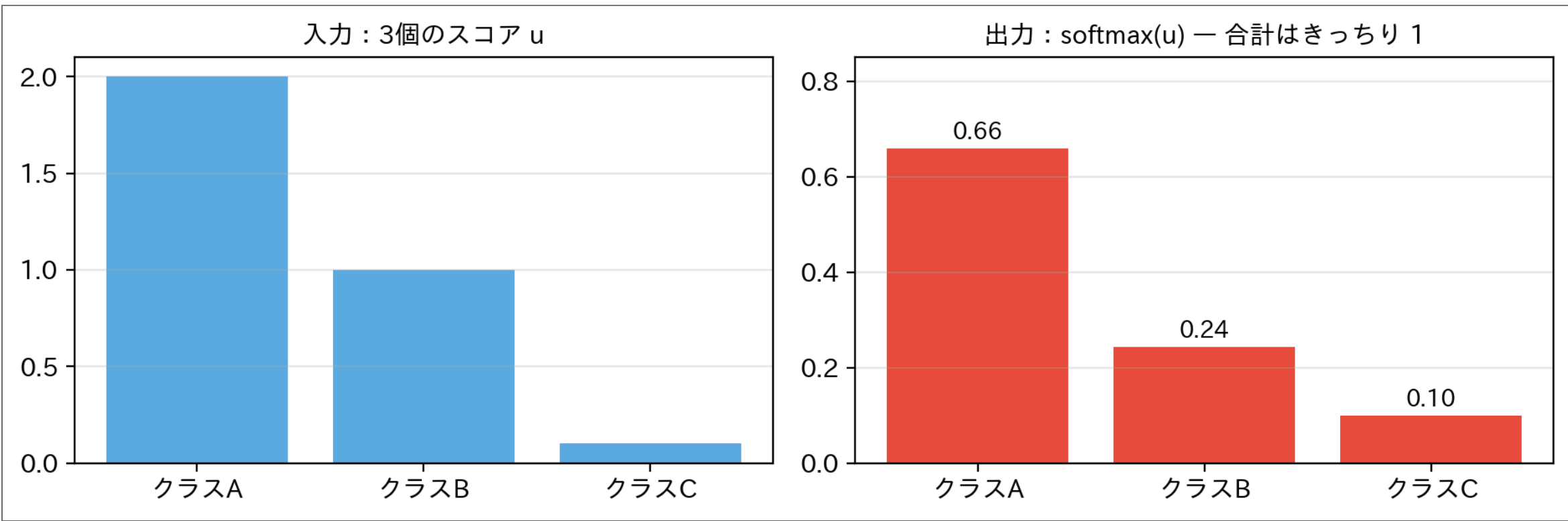
問題	出力の活性化関数
2クラス分類	シグモイド（1個の確率）
多クラス分類	ソフトマックス（10個の確率分布）

$\mathrm{softmax}(u_k) = \frac{e^{u_k}}{\sum_j e^{u_j}} \quad \text{10個の出力を「合計1の確率」に変換}$

- 損失は多クラス版の交差エントロピー：正解クラスの確率 $p_{\text{正解}}$ に対して $-\log p_{\text{正解}}$ （あの $-\log$ のカーブ の10クラス版）
- PyTorch では **nn.CrossEntropyLoss** がソフトマックスごとやってくれる（打ち消しの恩恵もそのまま）

5

ソフトマックスの仕事ぶり



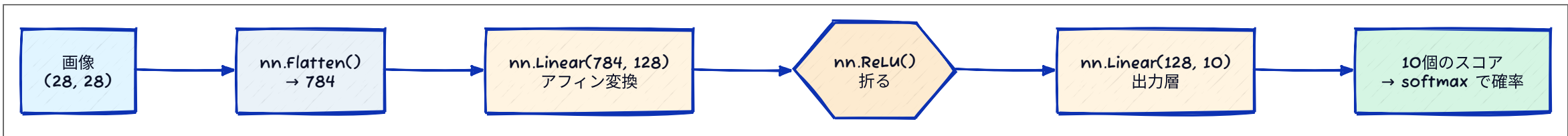
- スコアが大きいほど確率も大きい。ただし e^u を通すので **差が強調される**（スコア差 1.0 が、確率では 0.66 対 0.24）
- 合計 1 の「自信の配分」になる — 10クラスなら10個のスコアを10個の確率に

6

モデル：見慣れた3層

```
Code
1 torch.manual_seed(0)
2
3 model = nn.Sequential(
4     nn.Flatten(),          # (28,28) → 784
5     nn.Linear(784, 128),   # アフィン変換
6     nn.ReLU(),            # 折る
7     nn.Linear(128, 10),    # 出力層 (10クラスぶんのスコア)
8 )
9
10 n_params = sum(p.numel() for p in model.parameters())
11 print(f"パラメータ総数: {n_params:,}")
```

Result
パラメータ総数: 101,770



- 魚の最小単位は**3個**だった。いまや**10万個** — それでも学習ループは同じ3拍子

i Adam — 最適化回の伏線回収

勾配降下法の回で名前だけ出た「学習率を自動で加減する改良版」。中身は勾配降下法そのもの（勾配の使い方が少し賢いだけ）。大きなネットでは標準装備。

ミニバッチ：6万枚は一口ずつ

- 6万枚まとめて勾配を計算すると重い → **128枚ずつ** 取り出して1歩ずつ進む — この一口が**ミニバッチ**（mini-batch）
- データを一巡することを **1エポック**（epoch）と呼ぶ

```
Code
1 train_loader = torch.utils.data.DataLoader(train_ds, batch_size=128, shuffle=True)
2 test_loader  = torch.utils.data.DataLoader(test_ds,  batch_size=1000)
3
4 loss_fn      = nn.CrossEntropyLoss()
5 optimizer    = torch.optim.Adam(model.parameters(), lr=1e-3)
```

- NN最小単位の回で 「魚を1匹ずつ見せる」 とやったのと同じ発想（あちらはバッチサイズ1）

学習する

Code

```
1 for epoch in range(1, 4):
2     for x_batch, y_batch in train_loader:
3         optimizer.zero_grad()
4         loss = loss_fn(model(x_batch), y_batch)
5         loss.backward()
6         optimizer.step()
7
8     # テストデータ（見ていない1万枚）で採点 - 過学習と汎化の回の鉄則
9     correct = 0
10    with torch.no_grad():
11        for x_batch, y_batch in test_loader:
12            correct += (model(x_batch).argmax(dim=1) == y_batch).sum().item()
13    print(f"エポック {epoch} | テスト正答率: {correct / len(test_ds):.2%}")
```

Result

エポック 1 | テスト正答率: 93.50%
エポック 2 | テスト正答率: 95.14%
エポック 3 | テスト正答率: 96.33%

- 見たことのない手書き1万枚を、**96%以上**の精度で読めるようになった

Tweak

どこまで賢くできるか



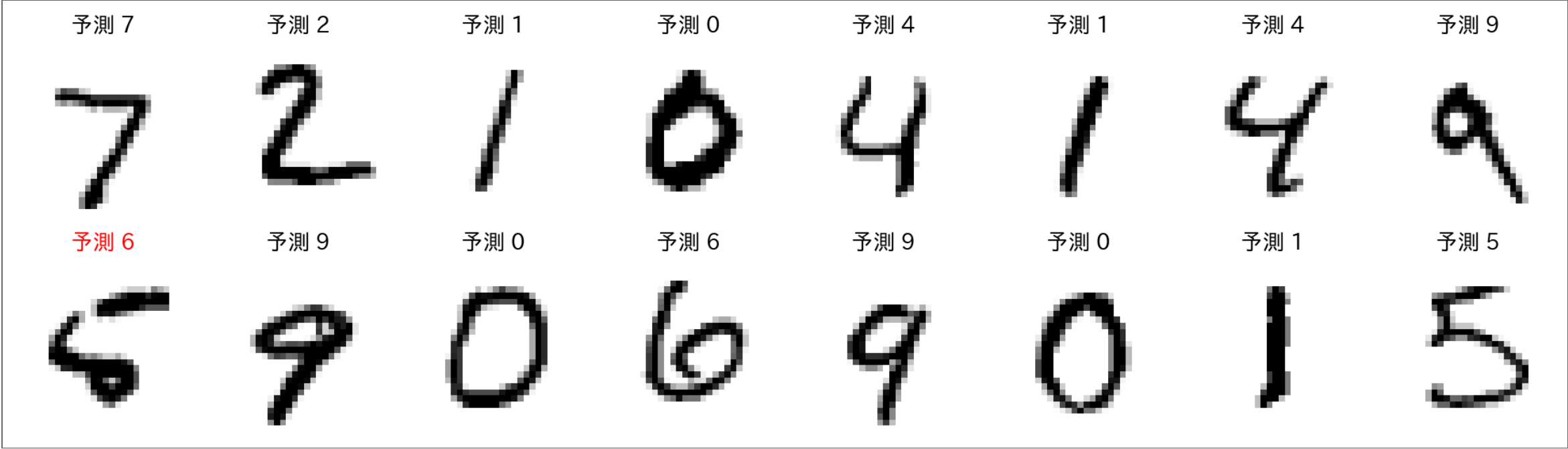
- モデル定義と学習ループ を書き換えて実行（学習に数分かかる変更もある）

1	長く学習させる	<code>range(1, 4) → range(1, 9)</code>
2	中間を太くする	<code>nn.Linear(784, 128)</code> と <code>nn.Linear(128, 10)</code> の <code>128</code> → <code>512</code>
3	中間をもう1段重ねる	<code>nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, 128), nn.ReLU(), nn.Linear(128, 10)</code>

💡 ここで観察

- どの変更が「テスト正答率1%」あたりのコスパが良い？ 97%の壁、98%の壁はどこで破れる？
- 伸びが止まったら**訓練データの正答率**も出して比べてみる — ギャップが開き始めたら、それは見覚えのある現象（過学習）

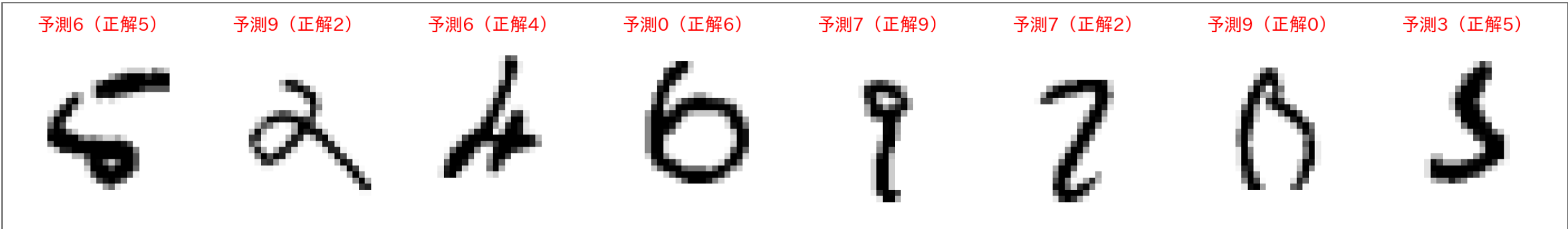
読めている様子



- 予測ラベルは **モデルの出力**。初回に「謎」として見せた光景が、いま手元のコードで起きている

11

間違いを眺める



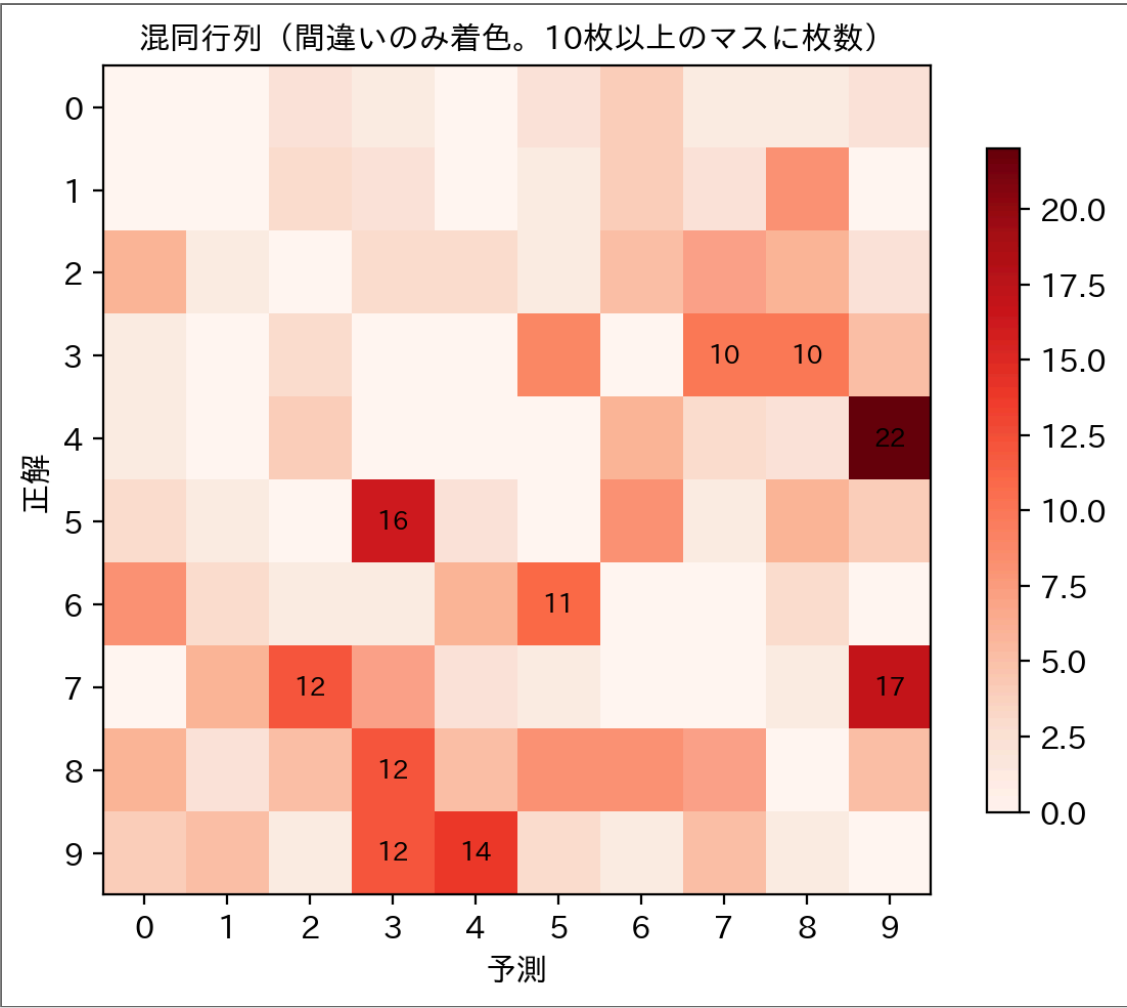
- 間違えた字をよく見ると、**人間でも迷うものが多い**
- 精度の数字だけでなく **間違いの中身** を見るのが、モデルと付き合うコツ（混同行列の発想）

12

どの数字とどの数字が混ざるのか

① クイズ

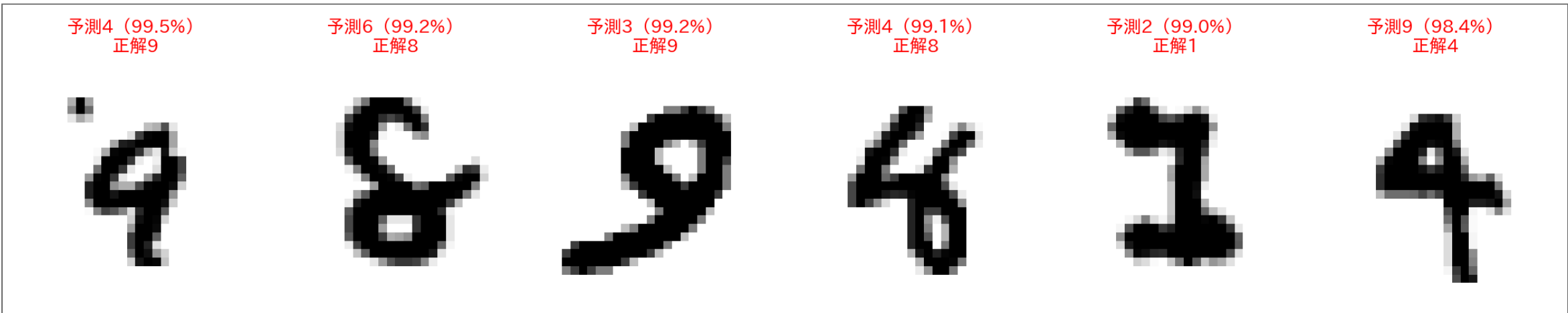
0～9 のうち、この機械が**いちばん混同しやすいペア**はどれだと思う？



- 過学習と汎化の回の混同行列は2×2だった。これはその**10クラス版**
- 最多は **4→9（22枚）**、逆向きの 9→4 も14枚 — **4と9はお互いに紛らわしい**。次点は 7→9・5→3
- 間違いはランダムに散らばらない。**形の似たペア**に集中する

13

自信満々の間違い

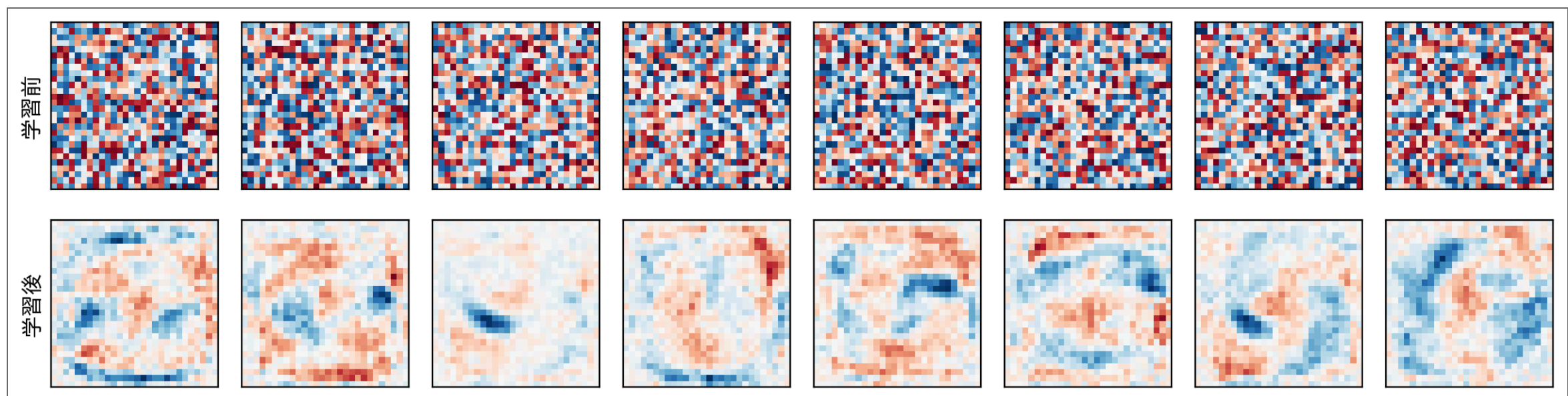


- 最上位は「9」を **99.5%の自信**で「4」と誤読 — 精度96%の機械は、間違えるときも自信満々のことがある
- **-log のカーブ** の左端そのもの。学習中ならこの1枚が**莫大な損失**を出して重みを大きく動かす
- 確率はあくまで**モデルの主観**。「自信がある＝正しい」ではない

14

育った重みを覗く

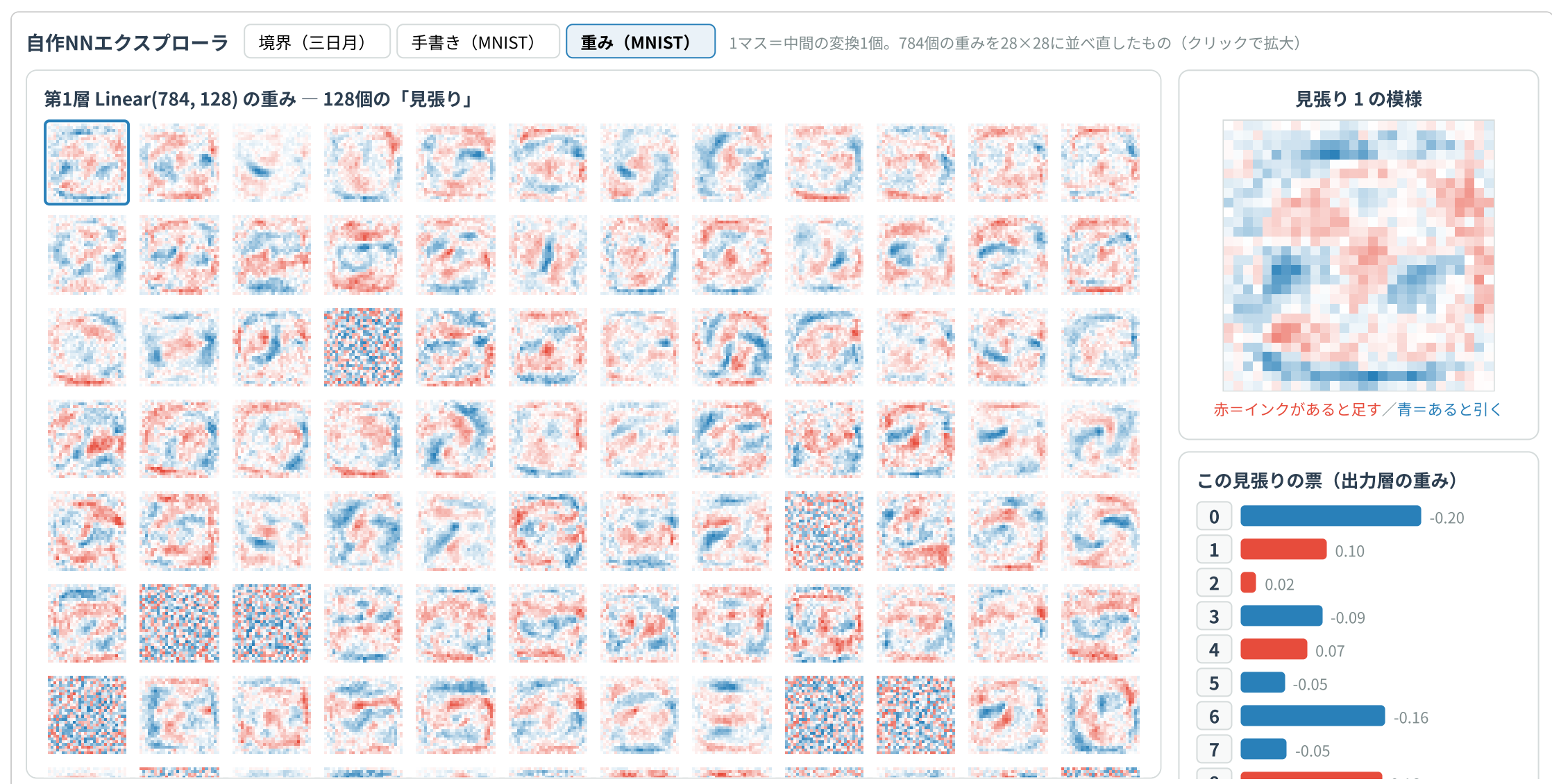
- 第1層 `nn.Linear(784, 128)` の各ユニットは、784本の重みを持つ = **28×28の「模様」** として絵にできる



- 赤い場所にインクが乗ると足し、青い場所なら引く — その合計が ReLU の入り口 z
- 学習前（同じシードで初期化し直したもの）は一様な砂嵐。学習後は**中央にインクの通り道の模様**が育っている
- 魚のとき 学習された α を読んだ のと同じことを、10万パラメータでやっただけ

15

128人の見張りを全員閲覧する



エクスプローラを別タブで開く

- 1マス＝中間ユニット1個の模様（さっきの図の**128人全員版**）。クリックで拡大
- 右の票バー＝そのユニットが反応したとき、**どの数字に票が入るか**（出力層の重み）
- お気に入りを1人選んで、「こいつは何を探している係か」を隣の人に説明してみよう

16

その場で書いて、読ませる

自作NNエクスプローラ

境界（三日月）

手書き（MNIST）

重み（MNIST）

マウスやタッチで数字を書く — 講義で学習させた本物の重みが判定する

消す 太く・大きく書くのがコツ（MNISTの字は線が太い）

モデルに見えている
28×28

切り出し → 長辺20px → 重心を中央へ
(MNISTと同じ揃え方)

10個の出力率に

テスト1万枚の学習の実重み

本物のMNISTを流す

0

1

2

3

4

5

6

7

8

9

4→9

6→8

3→9

4→8

2→1

9→4

薄赤枠＝この機械が読み間違えるテスト画像（予測→正解）

エクスプローラを別タブで開く

- 今日学習させた**実物の重み**（テスト正答率 96.3%）がブラウザの中で動いている。マウスで数字を書いてみる
- 「モデルに見えている28×28」の枠に、MNIST と同じ揃え方（切り出し→縮小→センタリング）を通った姿が映る
- 下の見本ボタンは本物のテスト画像。**薄赤枠の6枚はこの機械が誤読した字** — 君は正しく読める？

自分の字は読めるのか（持ち帰りデモ）

紙に数字を書いて撮影し、この機械に読ませてみよう。ただし**写真をそのまま28×28に縮めても、まず読めない** — MNISTの画像は「数字を20ピクセル角に収め、重心をど真ん中に置く」ように揃えてあり、撮った写真はそうならないから。同じ揃え方を自分でやってから見せる：

Code

```
1 from PIL import Image
2 import numpy as np
3 import torch
4
5 # 1. 読み込み：グレースケール化して「インク=1、紙=0」に正規化
6 a = np.array(Image.open("my_digit.png").convert("L"), dtype=np.float32)
7 a = (a.max() - a) / (a.max() - a.min())
8
9 # 2. MNISTと同じ揃え方：数字を切り出し → 長辺20pxに縮小 → 重心を中央へ
10 rows, cols = np.where(a > 0.3)
11 a = a[rows.min():rows.max()+1, cols.min():cols.max()+1]
12 h, w = a.shape
13 s = 20.0 / max(h, w)
14 img = Image.fromarray((a * 255).astype(np.uint8)).resize((round(w*s), round(h*s)))
15 a = np.array(img, dtype=np.float32) / 255.0
16 x28 = np.zeros((28, 28), dtype=np.float32)
17 h, w = a.shape
18 x28[(28-h)//2:(28-h)//2+h, (28-w)//2:(28-w)//2+w] = a
19 ys, xs = np.mgrid[0:28, 0:28]
20 cy, cx = (x28*ys).sum()/x28.sum(), (x28*xs).sum()/x28.sum()
21 x28 = np.roll(x28, (round(14-cy), round(14-cx)), axis=(0, 1))
22
23 # 3. 推論
24 x = torch.tensor(x28).unsqueeze(0).unsqueeze(0)
25 print("予測:", model(x).argmax(dim=1).item())
```

💡 うまく読めないとき

- **太いペンで、大きく** 書く（MNISTの字は線が太い）。紙は白く、明るい場所で撮る
- それでも読めない字は「間違いを眺める」の仲間入り — 訓練データの字と自分の字の**分布の違い**が原因
- 精度96%とは「**100枚中4枚は間違える**」という意味でもある。1枚読ませて外れても、それはこの機械の正直な実力

約束は果たされたか：部品の棚卸し

この機械の中に、説明できない部品が残っていないか確認する。

部品	正体	学んだ回
<code>nn.Linear</code>	アフィン変換（傾きと切片の一般化）	決定境界／NN最小単位
<code>nn.ReLU</code>	空間を折る非線形	変換を重ねる
ソフトマックス+交差エントロピー	微分しやすい損失	NN最小単位／今回
<code>loss.backward()</code>	連鎖律の自動化	バックプロパゲーション
<code>Adam</code>	勾配降下法の改良版	最適化と勾配降下法
テストデータで採点	汎化を測る鉄則	過学習と汎化

① 約束の回収

ブラックボックス、ゼロ。 全部品を手で作るか、手計算で検算済み。

19

到達チェック

96%の中身を読む



「テスト正答率96.3%」だけでは分からないこのモデルの性質を、前の観察から3つ挙げよ。

- 0. 混ざりやすい数字の組
- 1. 自信満々で間違う例
- 2. 自分の手書き画像で成績が変わる理由

20

この先の世界

- **画像がもっと大きく複雑になったら？**
→ 784次元を全部つなぐのは無駄が多い → 近くのピクセルだけ見る **畳み込み（CNN）**
- **入力が文章だったら？**
→ 単語同士の関係に重みをつける **Attention** → **Transformer** → いまの **大規模言語モデル（LLM）**
- どちらも今日の機械と同じ原理の上に立つ：**変換を重ねて、損失を決めて、勾配で学習する**

💡 ヒント

名前は新しくても、開け方はもう知っている。「forward は何か」「損失は何か」「勾配はどう流れるか」 — この3つを聞けばいい。

21

講義全体のまとめ

- 魚の選別から始まり、**fit()** という1行の中身を開け続けた
 - しきい値 → 直線 → 木 → ロジスティック回帰 → 多層NN
 - 損失（交差エントロピー・MSE）と勾配降下法が全編を貫く背骨
- 満点を疑うこと（過学習）、スケールを揃えること、テストで採点すること
- 最後に、**手書き数字を読む機械**を自分の部品で組み上げた

① 持ち帰り

「自分の字は読めるのか」のデモを、ぜひ家で。読めたら勝ち、読めなくても **なぜかを説明できる** — それがこの講義の到達点。